

Physics-Informed Neural Networks (PINNs)

The 1D Heat Diffusion Equation

IMW 2026 – Deep Learning Track

1 Motivation

Every model in the preceding sequence of this workshop – the perceptron, the multilayer perceptron (MLP), the convolutional neural network (CNN), and the recurrent architectures (RNN/LSTM) – learns a mapping f_θ purely from labeled data $\{(x_i, y_i)\}_{i=1}^N$ by minimizing an empirical loss such as

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_\theta(x_i) - y_i\|^2. \quad (1)$$

This is powerful, but it has a well-known weakness in scientific and engineering applications: labeled data is often scarce, expensive, or noisy (a single borehole temperature sensor, a handful of seismic stations, a sparse set of GPS deformation points), while the *governing physical law* is frequently known exactly, in the form of a partial differential equation (PDE).

A **Physics-Informed Neural Network** (PINN), introduced by Raissi, Perdikaris & Karniadakis (2019), folds that governing law directly into the training objective. The network is still an ordinary MLP; what changes is the loss function, which now penalizes violations of the PDE, the initial conditions (IC), and the boundary conditions (BC), evaluated at points sampled from the domain – with no labeled solution data required at all in the purely *forward* problem.

2 The governing equation

We study the **1D heat (diffusion) equation**, a linear parabolic PDE:

$$\frac{\partial u}{\partial t}(x, t) = \alpha \frac{\partial^2 u}{\partial x^2}(x, t), \quad x \in [0, L], \quad t \in [0, T], \quad (2)$$

where $u(x, t)$ is temperature and $\alpha > 0$ is the thermal diffusivity of the medium. Physically, Eq. (2) describes how heat spreads through a 1D slice of material: a natural analogue is the conductive cooling of a thin lava dike or dome, where an initially hot rock body loses heat to cooler surroundings until it reaches thermal equilibrium.

2.1 Initial and boundary conditions

To make the problem well-posed we impose:

$$u(x, 0) = \sin(\pi x), \quad (\text{initial condition, IC}) \quad (3)$$

$$u(0, t) = 0, \quad u(L, t) = 0, \quad (\text{Dirichlet boundary conditions, BC}) \quad (4)$$

with $L = 1$. Physically, Eq. (3) is the initial temperature profile (hot in the middle, zero at the edges), and Eq. (4) says both ends of the domain are held at a fixed reference temperature (a conductive/cold boundary) for all time.

2.2 Exact solution (for validation only)

This particular IC/BC pair admits a closed-form solution by separation of variables:

$$u(x, t) = \sin(\pi x) e^{-\alpha \pi^2 t}. \quad (5)$$

In a real research problem we would not have Eq. (5) available – that is precisely why we need a PINN (or a numerical solver) in the first place. Here we keep it deliberately so that we can rigorously check whether the trained network has learned the actual physics, rather than having merely reduced a loss number.

3 From PDE to a trainable loss function

Let $u_\theta(x, t)$ denote the neural network’s output, a function of two scalar inputs (x, t) and one scalar output, with trainable parameters θ (weights and biases). Define the **PDE residual**:

$$f_\theta(x, t) := \frac{\partial u_\theta}{\partial t}(x, t) - \alpha \frac{\partial^2 u_\theta}{\partial x^2}(x, t). \quad (6)$$

If u_θ exactly satisfied Eq. (2), then $f_\theta \equiv 0$ everywhere. The key technical trick of a PINN is that $\partial u_\theta / \partial t$ and $\partial^2 u_\theta / \partial x^2$ are computed by **automatic differentiation** of the network with respect to its own inputs (not the weights) – exactly the same reverse-mode differentiation machinery (`tf.GradientTape`, `torch.autograd`) used to compute gradients with respect to weights during backpropagation, just applied to x and t instead. No mesh, no finite-difference stencil is needed.

We then define three loss terms, each a mean squared error evaluated on a set of sampled points:

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_f} \sum_{j=1}^{N_f} |f_\theta(x_f^{(j)}, t_f^{(j)})|^2, \quad \{(x_f^{(j)}, t_f^{(j)})\}_{j=1}^{N_f} - \text{interior collocation points}, \quad (7)$$

$$\mathcal{L}_{\text{IC}}(\theta) = \frac{1}{N_0} \sum_{j=1}^{N_0} |u_\theta(x_0^{(j)}, 0) - \sin(\pi x_0^{(j)})|^2, \quad \{x_0^{(j)}\}_{j=1}^{N_0} - \text{points on } t = 0, \quad (8)$$

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_b} \sum_{j=1}^{N_b} \left(|u_\theta(0, t_b^{(j)})|^2 + |u_\theta(L, t_b^{(j)})|^2 \right), \quad \{t_b^{(j)}\}_{j=1}^{N_b} - \text{points on } x = 0, L. \quad (9)$$

The total training objective is a weighted sum:

$$\mathcal{L}(\theta) = w_f \mathcal{L}_{\text{PDE}}(\theta) + w_0 \mathcal{L}_{\text{IC}}(\theta) + w_b \mathcal{L}_{\text{BC}}(\theta), \quad (10)$$

minimized over θ with a standard first-order optimizer (Adam), and optionally refined with a quasi-Newton method (L-BFGS) once the loss plateaus. In this exercise we take $w_f = w_0 = w_b = 1$ as a baseline; Exercise B below asks you to investigate the effect of unequal weights.

3.1 Why tanh, not ReLU

Because $\mathcal{L}_{\text{PDE}}$ requires a *second* derivative of the network output with respect to x , the activation function must be at least twice differentiable with a non-trivial second derivative. ReLU is piecewise linear, so $\partial^2 u_\theta / \partial x^2 = 0$ almost everywhere when ReLU is used, which destroys the physics loss signal. Smooth activations (`tanh`, `sin`, `swish`) are therefore standard in PINN architectures.

4 Network architecture

A fully-connected feed-forward network (MLP), identical in spirit to the MLP notebook earlier in this workshop, but with:

- **Input layer:** 2 units, (x, t) ;
- **Hidden layers:** typically 4–8 layers of 20–50 units each, `tanh` activation, Glorot-normal initialization;
- **Output layer:** 1 unit, linear activation, giving $u_\theta(x, t)$.

5 Algorithm summary

1. Sample N_f interior collocation points, N_0 initial-condition points, N_b boundary points.
2. Forward-pass the network to get u_θ at all sampled points.
3. Use automatic differentiation to compute u_t and u_{xx} at the collocation points, forming the residual f_θ (Eq. 6).
4. Compute $\mathcal{L}_{\text{PDE}}$, $\mathcal{L}_{\text{IC}}$, $\mathcal{L}_{\text{BC}}$ and sum them (Eq. 10).
5. Backpropagate through *all* of the above (including the nested derivatives) to get $\nabla_\theta \mathcal{L}$, and take an optimizer step.
6. Repeat for many epochs; monitor each loss component separately.
7. Validate against the exact solution, Eq. (5), on a dense grid never used during training.

6 Exercises

Work through these using the accompanying `backbone` notebook, which provides the data-sampling and plotting code but leaves the core PINN logic (network construction, residual computation, loss assembly, training loop) as guided TODO blocks for you to complete.

Exercise A. Sensitivity to collocation density. Vary $N_f \in \{100, 500, 2000, 5000\}$. Plot the maximum absolute error against u_{exact} as a function of N_f . At what point do additional collocation points stop improving accuracy? Relate your answer to the PDE residual loss curve.

Exercise B. Loss weighting. Introduce weights (w_f, w_0, w_b) in Eq. (10) and try $(1, 10, 10)$ and $(1, 100, 100)$. How does over- or under-weighting the IC/BC terms affect the solution near $t = 0$ versus in the interior? This is a known PINN pathology; see Wang, Teng & Perdikaris (2021).

Exercise C. Inverse problem: parameter estimation. Treat α as an unknown, trainable scalar rather than a fixed constant. Add a small set of noisy synthetic “sensor” observations of u , drawn from Eq. (5) plus Gaussian noise, as an additional data-fit loss term $\mathcal{L}_{\text{data}}$. Recover α and compare it to the true value $\alpha = 0.1$. This mirrors a real scenario: estimating the thermal diffusivity of volcanic rock from a handful of borehole temperature sensors.

Exercise D. A nonlinear PDE: Burgers’ equation. Replace Eq. (2) with the viscous Burgers’ equation

$$u_t + u u_x = \nu u_{xx}, \quad \nu = \frac{0.01}{\pi}, \quad (11)$$

on $x \in [-1, 1]$, with $u(x, 0) = -\sin(\pi x)$ and $u(-1, t) = u(1, t) = 0$. This is the canonical benchmark of Raissi et al. (2019) and develops a sharp, evolving shock. Observe how much harder training becomes, and what mitigations help (denser collocation points near the shock, deeper network, adaptive loss weights).

Exercise E. Comparison against a classical numerical baseline. Implement a simple explicit finite-difference (FTCS) scheme for Eq. (2). Compare wall-clock time, accuracy, and

memory footprint against the PINN. Discuss when a PINN is genuinely advantageous (irregular domains, inverse problems, high dimensionality) versus when a classical numerical method wins outright (this well-posed, low-dimensional, linear case).

References

- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.
- Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*.
- Karniadakis, G. E., et al. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3, 422–440.